AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science

Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Spring 2025-2026

Section: A

Group No: 02

Project Report On

Project Name:

PeerSync

Connect, Collaborate, Succeed

Supervised By

Tofayet Sultan

Submitted By:

Name	Id
Sajal Mollick	22-49653-3
Azraful Alam	24-57476-2
Azizul Alam	22-47521-2
Sabrina Islam Shorna	22-49569-3

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				**Total =**
Requirement fulfillment	**Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.**				
Validation	**Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.**				
Verification	**Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.**				

1. Introduction

PeerSync is a clever peer-matching and team managing system created for university students to assist them in discovering appropriate teammates for group projects and collaborative tasks. Nowadays students' collaboration has become an indispensable element in the academic learning particularly in the technical and engineering areas. Nevertheless students frequently encounter problems when it comes to selecting dependable and competent teammates. Most teams are either formed randomly or through personal friendships which may cause to problems in communication, unbalanced task distribution, lack of responsibility, and low quality of project. PeerSync came into existence to address these frequently occurring problems by establishing a structured platform for students to connect according to their skills, interests, working styles, and reputation.

PeerSync's concept is basically inspired by social matching apps, where people browse student profiles and swipe right if they want to connect or left if not. The system implements a swipe-based matching method that enables a connection only when both parties show mutual interest. It makes team formation easier and more efficient. Besides matching, the platform offers reputation tracking via peer ratings, giving students an opportunity to grade teammates on contribution, communication, reliability, and punctuality. Accountability is facilitated by these features and they also guide decision-making before a team is formed in a project context.

PeerSync was created with the frontend interface C# Windows Forms, ADO.NET was utilised for connecting to the database, and for secure data storage, Microsoft SQL Server was the platform of choice. The software gives access to two types of roles, namely student and admin. Students are able to open their profiles, add skills and work preferences at the same time, do group formation for projects, and rate their peers after finishing projects. On the other hand, admins have a whole dashboard dedicated to them where they can see user activities, handle accounts, and make sure the platform works properly. Furthermore, the database has been normalized so as to keep the data accurate, get rid of duplicates, and also make operations fast and efficient.

The initiative targets boosting students' collaboration at universities by streamlining how teams are formed and encouraging students to maintain a professional attitude in their academic projects. PeerSync, through the use of smart matching, a single place for handling groups, and a clear method of giving feedback, offers an effective answer to the issues commonly encountered in settings where learning is done through groups. The platform is a source of great help for the students by allowing them to gain more time and also raise the standard of teamwork. In addition, universities get a tool to promote peer collaboration and enhance accountability.

2. ER Diagram

The Entity Relationship (ER) Diagram below illustrates the data model of PeerSync, showing the entities, their attributes, and the relationships between them. The diagram captures the core tables including USER, SKILL, WORK_STYLE, SWIPE, MATCH, PROJECT_GROUP, GROUP_MEMBER, and RATING, along with their associations.

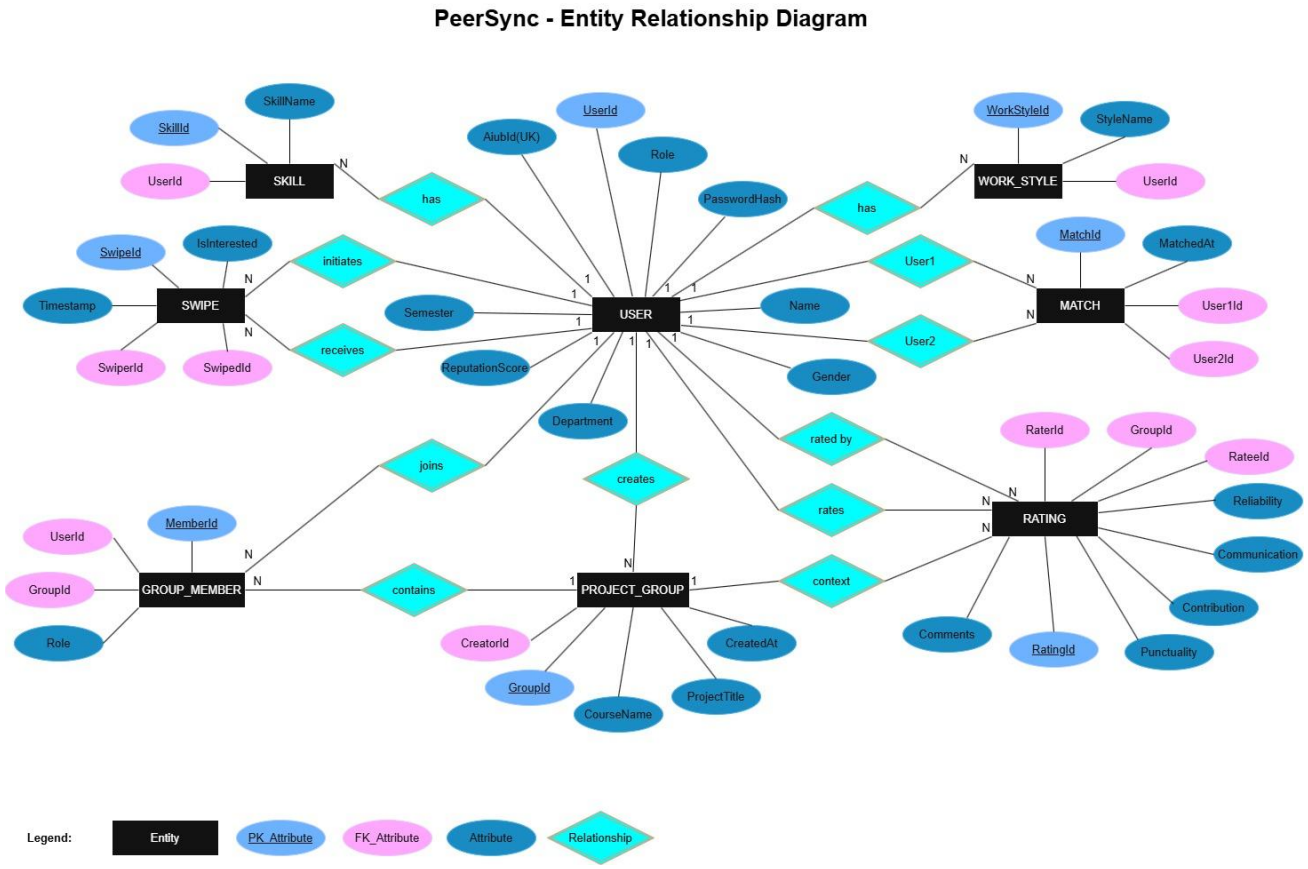

Figure 1: PeerSync ER Diagram

3. Normalization – Final Table Structure

The PeerSync database follows Third Normal Form (3NF). Primary Keys (PK) are underlined and bold. Foreign Keys (FK) are shown in bold dark blue. All tables are free from partial and transitive dependencies.

Table 1: USER

S/N	Attribute	Data Type / Constraint
1	**UserId (PK)**	INT, IDENTITY, PRIMARY KEY
2	AiubId (UK)	NVARCHAR(20), UNIQUE NOT NULL
3	PasswordHash	NVARCHAR(255), NOT NULL
4	Name	NVARCHAR(100), NOT NULL
5	Gender	NVARCHAR(10)
6	Department	NVARCHAR(50)
7	Semester	NVARCHAR(10)
8	Role	NVARCHAR(20), DEFAULT 'Student'
9	ReputationScore	INT, DEFAULT 0

Table 2: SKILL

S/N	Attribute	Data Type / Constraint
1	**SkillId (PK)**	INT, IDENTITY, PRIMARY KEY
2	**UserId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
3	SkillName	NVARCHAR(100), NOT NULL

Table 3: WORK_STYLE

S/N	Attribute	Data Type / Constraint
1	**WorkStyleId (PK)**	INT, IDENTITY, PRIMARY KEY
2	**UserId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
3	StyleName	NVARCHAR(100), NOT NULL

Table 4: SWIPE

S/N	Attribute	Data Type / Constraint
1	**SwipeId (PK)**	INT, IDENTITY, PRIMARY KEY
2	**SwiperId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
3	**SwipedId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
4	IsInterested	BIT, NOT NULL
5	Timestamp	DATETIME, DEFAULT GETDATE()

Table 5: MATCH

S/N	Attribute	Data Type / Constraint
1	**MatchId (PK)**	INT, IDENTITY, PRIMARY KEY
2	**User1Id (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
3	**User2Id (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
4	MatchedAt	DATETIME, DEFAULT GETDATE()

Table 6: PROJECT_GROUP

S/N	Attribute	Data Type / Constraint
1	**GroupId (PK)**	INT, IDENTITY, PRIMARY KEY
2	CourseName	NVARCHAR(100)
3	ProjectTitle	NVARCHAR(200)
4	**CreatorId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
5	CreatedAt	DATETIME, DEFAULT GETDATE()

Table 7: GROUP_MEMBER

S/N	Attribute	Data Type / Constraint
1	**MemberId (PK)**	INT, IDENTITY, PRIMARY KEY
2	**GroupId (FK)**	INT, FOREIGN KEY REFERENCES PROJECT_GROUP(GroupId)
3	**UserId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
4	Role	NVARCHAR(50)

Table 8: RATING

S/N	Attribute	Data Type / Constraint
1	**RatingId (PK)**	INT, IDENTITY, PRIMARY KEY
2	**RaterId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
3	**RateeId (FK)**	INT, FOREIGN KEY REFERENCES USER(UserId)
4	**GroupId (FK)**	INT, FOREIGN KEY REFERENCES PROJECT_GROUP(GroupId)
5	Contribution	INT (1-5)
6	Communication	INT (1-5)
7	Reliability	INT (1-5)
8	Punctuality	INT (1-5)
9	Comments	NVARCHAR(500)

4. Table Data

The following screenshots show sample data inserted into each of the eight PeerSync database tables to verify functionality and demonstrate realistic usage scenarios.

4.1 USER Table

	UserId	AiubId	PasswordHash	Name	Gender	Department	Semester	Role	ReputationScore
1	1	22-49653-3	12345	Sajal Mollick	Male	Cse	10	Student	4
2	2	22-47521-2	54321	Azizul Alam	Male	Cse	10	Student	3
3	3	24-57476-2	1234	Azraful Alam	Male	Cse	6	Student	4
4	4	22-49569-3	4321	Sabrina Islam	Female	Cse	10	Student	3

Figure 2: USER Table – Sample Data

4.2 SKILL Table

	SkillId	UserId	SkillName
1	1	1	C#
2	2	2	C#
3	3	3	C#
4	4	3	JAVA
5	5	4	C++
6	6	4	JAVA
7	7	4	C#

Figure 3: SKILL Table – Sample Data

4.3 WORK_STYLE Table

	WorkStyleId	UserId	StyleName
1	1	1	Night
2	2	2	Day
3	3	3	Day
4	4	4	Night

Figure 4: WORK_STYLE Table – Sample Data

4.4 SWIPE Table

	SwipeId	SwiperId	SwipedId	IsInterested	Timestamp
1	1	1	2	1	2026-05-16 00:03:20.220
2	2	1	3	1	2026-05-16 00:03:21.913
3	3	1	4	1	2026-05-16 00:03:22.863
4	4	2	1	1	2026-05-16 00:04:00.023
5	5	2	3	1	2026-05-16 00:04:02.907
6	6	2	4	1	2026-05-16 00:04:05.630
7	7	3	1	1	2026-05-16 00:04:45.367

Figure 5: SWIPE Table – Sample Data

4.5 MATCH Table

	MatchId	User1Id	User2Id	MatchedAt
1	1	2	1	2026-05-16 00:04:00.050
2	2	3	1	2026-05-16 00:04:45.383
3	3	3	2	2026-05-16 00:04:48.453
4	4	4	1	2026-05-16 00:05:17.953
5	5	4	2	2026-05-16 00:05:21.060
6	6	4	3	2026-05-16 00:05:23.643

Figure 6: MATCH Table – Sample Data

4.6 PROJECT_GROUP Table

	GroupId	CourseName	ProjectTitle	CreatorId	CreatedAt
1	1	OOP2	PeerSync	1	2026-05-16 00:06:46.923

Figure 7: PROJECT_GROUP Table – Sample Data

4.7 GROUP_MEMBER Table

	MemberId	GroupId	UserId	Role
1	1	1	1	Creator
2	2	1	2	Member
3	3	1	3	Member
4	4	1	4	Member

Figure 8: GROUP_MEMBER Table – Sample Data

4.8 RATING Table

	RatingId	RaterId	RateeId	GroupId	Contribution	Communication	Reliability	Punctuality	Comments
1	1	1	2	1	4	3	4	5	
2	2	1	3	1	3	4	2	5	
3	3	1	4	1	4	2	4	3	
4	4	2	1	1	4	4	3	4	
5	5	3	1	1	5	4	2	5	
6	6	4	1	1	5	5	5	5	
7	7	4	2	1	2	4	4	3	
8	8	4	3	1	5	5	5	5	

Figure 9: RATING Table – Sample Data

5. Features

PeerSync provides a comprehensive set of features designed to improve academic group collaboration:

5.1 Mutual Swiping

Students can browse candidate profiles on the Home Feed and either CONNECT or PASS. A match is only created when both students mutually express interest, ensuring team formations are always consensual and based on shared intent.

5.2 AIUB Student ID Verification

During registration, students must provide their valid AIUB Student ID. The system enforces uniqueness of the AIUB ID in the database, preventing duplicate or fake accounts and ensuring that only legitimate AIUB students can access the platform.

5.3 Anonymous Post-Project Ratings

After completing a project, group creators can rate their teammates across four dimensions: Contribution, Communication, Reliability, and Punctuality. Ratings encourage honest and fair feedback without fear of social repercussions.

5.4 Reputation Score System

Each student maintains a Reputation Score automatically calculated as the average of all ratings received across past projects. This score is visible on student profiles and the Home Feed, helping peers make informed decisions when selecting teammates.

5.5 Group Creation

Matched students can create project groups directly within the platform by specifying a course name, project title, and selecting matched peers as members. Group data is stored using a database transaction to ensure atomicity and data integrity.

5.6 Admin Dashboard

The Admin Dashboard provides full control over the student user base. Admins can search for students by name or AIUB ID, add new students, update existing records, and delete accounts using a cascaded transaction that safely removes all associated data.

6. Forms

The following screenshots illustrate the user interface forms of PeerSync. Each form serves a specific functional purpose within the application workflow.

6.1 Welcome Form

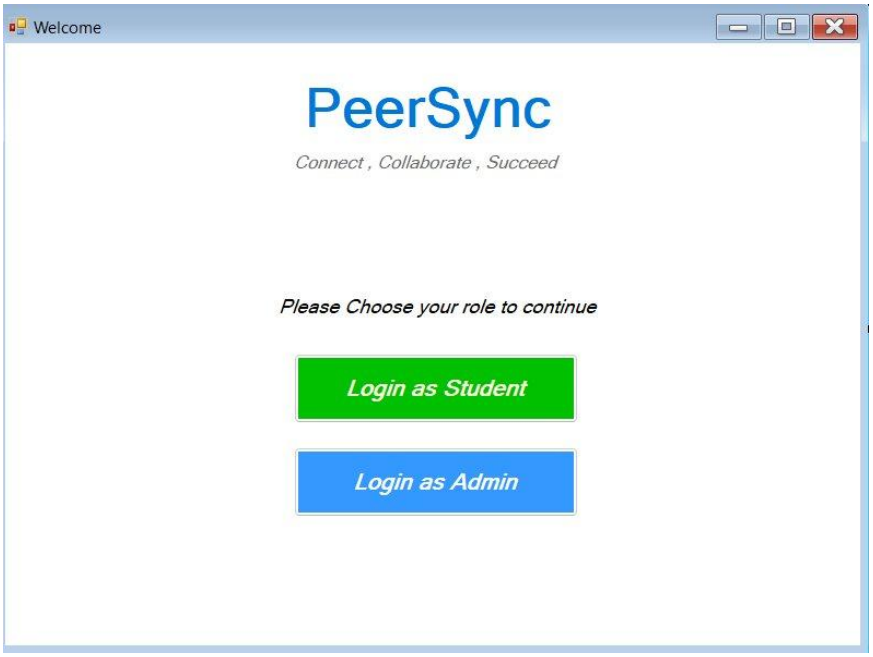

Figure 10: Welcome Form – Role Selection

6.2 Student Login Form

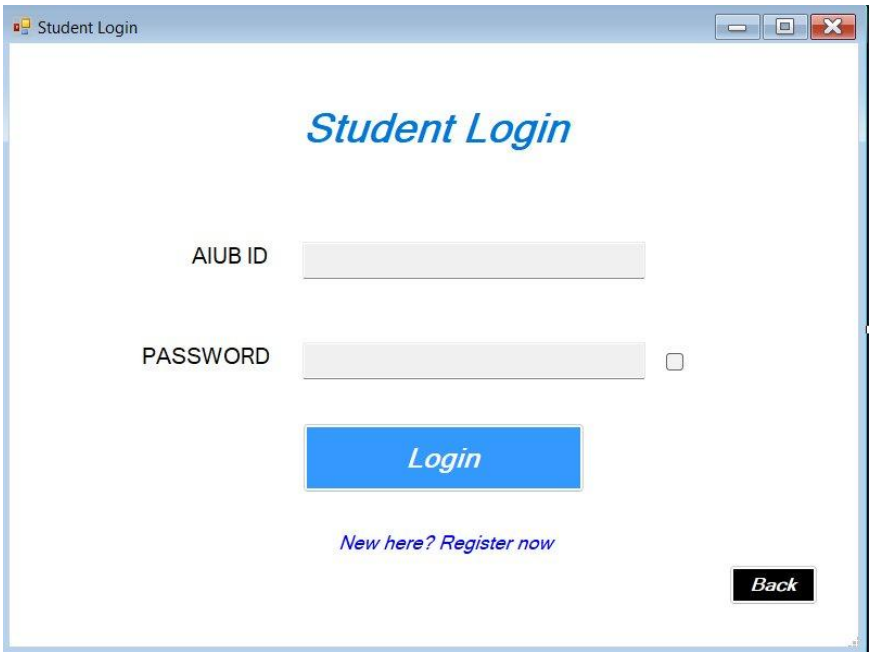

Figure 11: Student Login Form

6.3 Registration Form

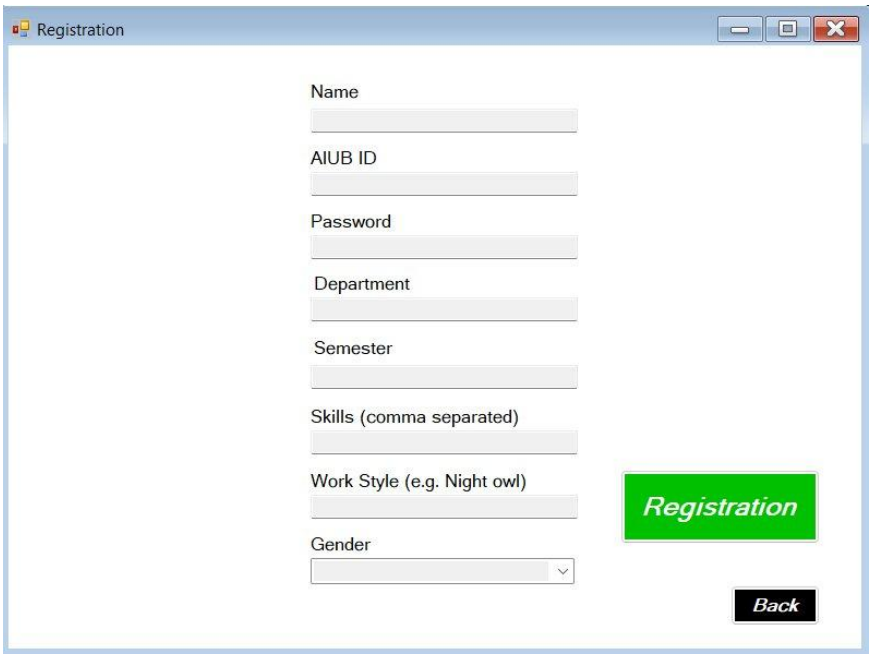

Figure 12: Student Registration Form

6.4 Home Feed Form

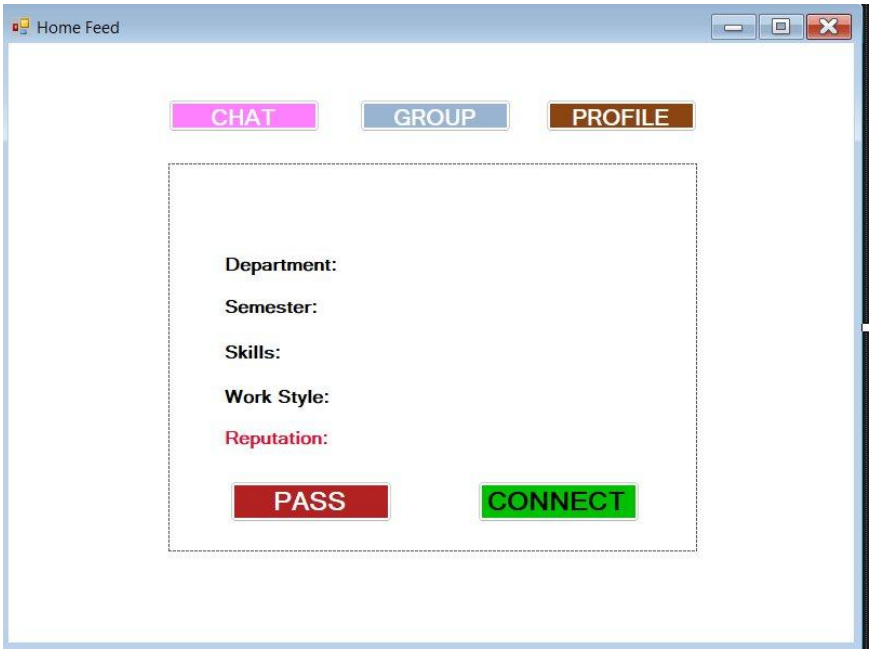

Figure 13: Home Feed – Swipe Interface

6.5 Group Form

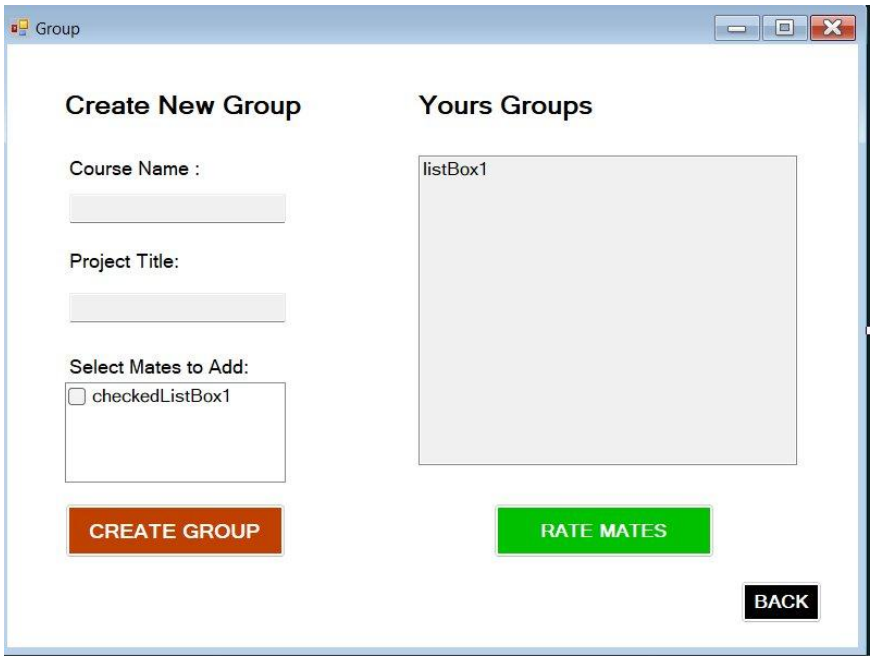

Figure 14: Group Creation and Management Form

6.6 Rate Team Mate Form

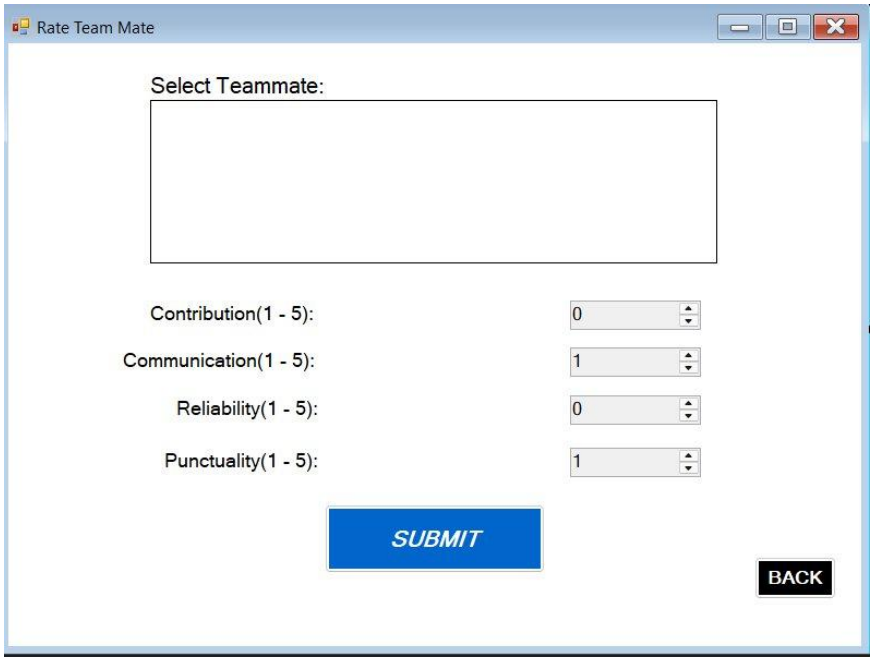

Figure 15: Rate Team Mate Form

6.7 My Profile Form

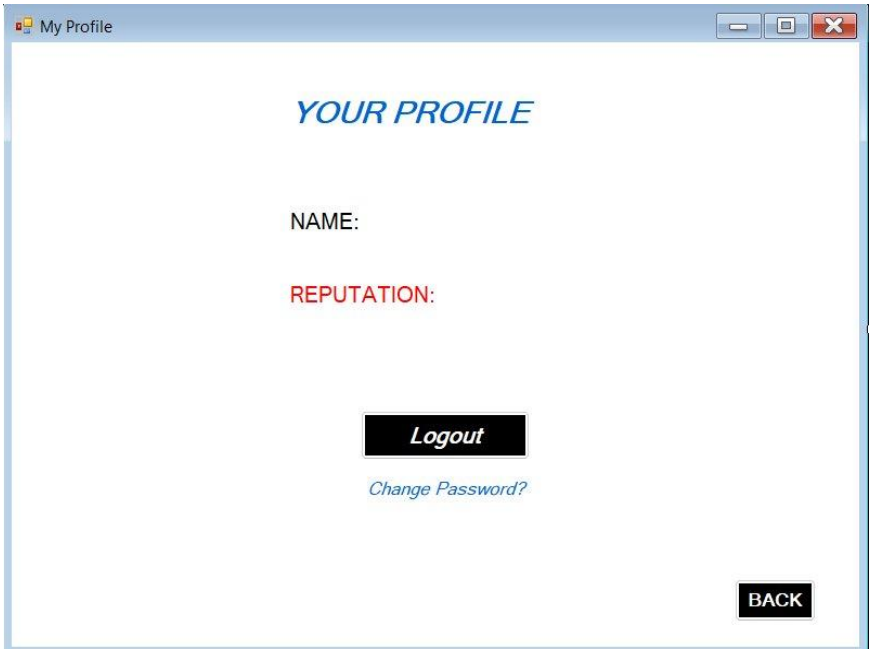

Figure 16: My Profile Form

6.8 Change Password Form

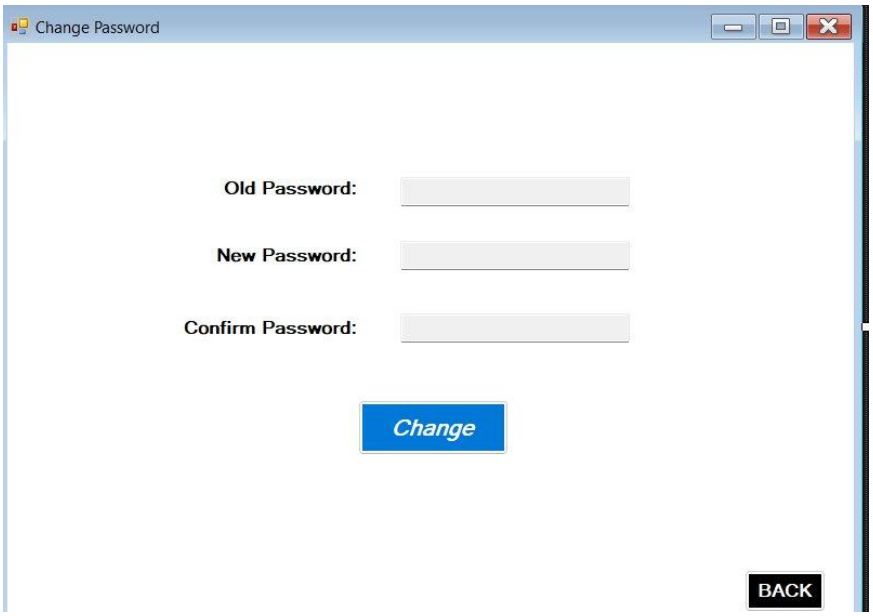

Figure 17: Change Password Form

6.9 Connection / Chat Form

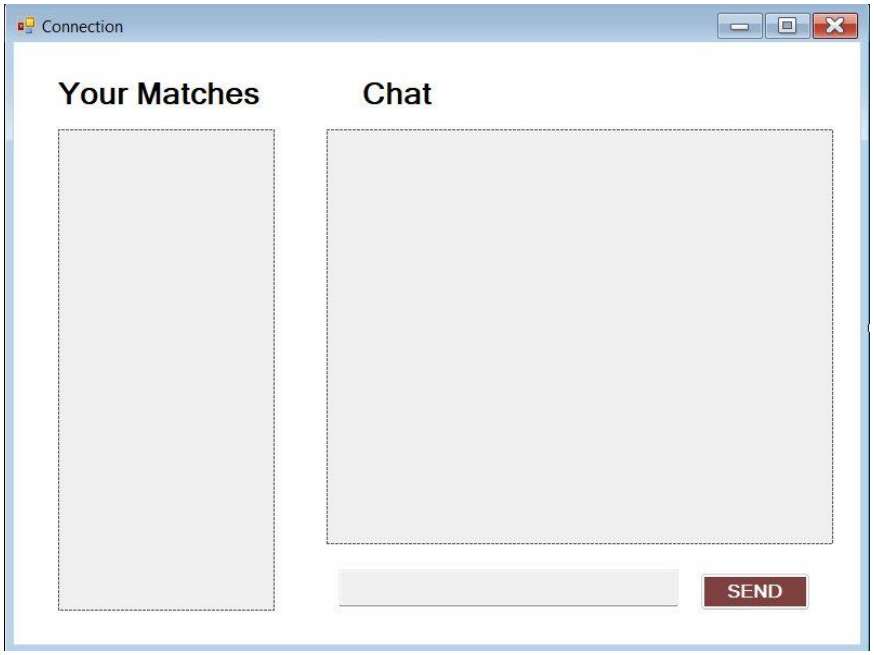

Figure 18: Connection and Chat Form

6.10 Admin Login Form

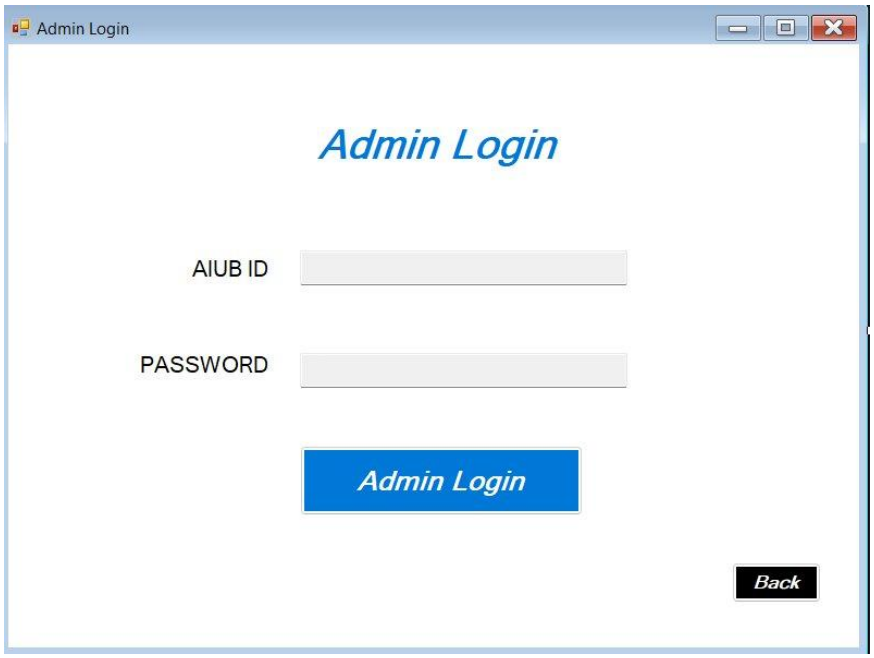

Figure 19: Admin Login Form

6.11 Admin Dashboard Form

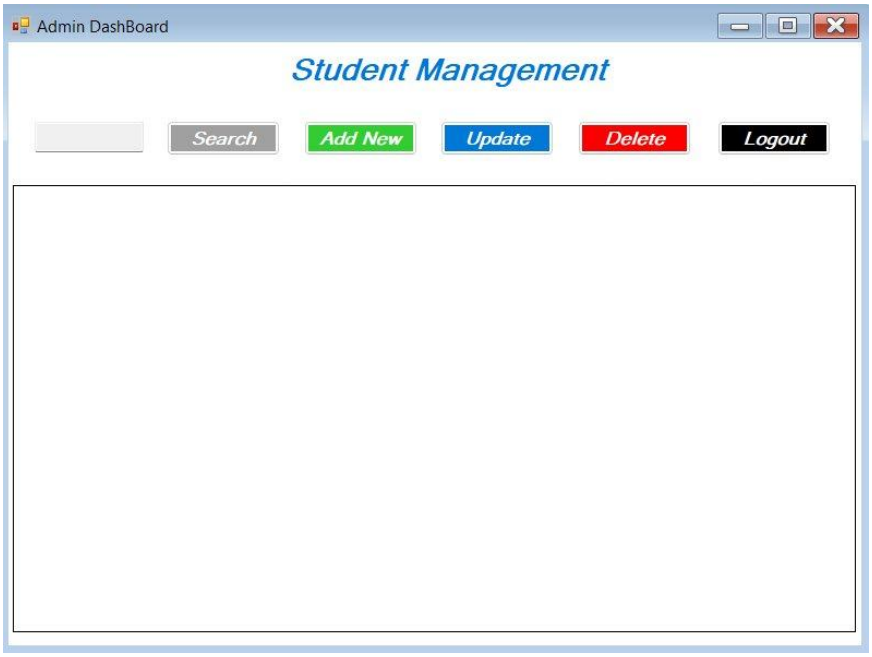

Figure 20: Admin Dashboard – Student Management

6.12 Update Form

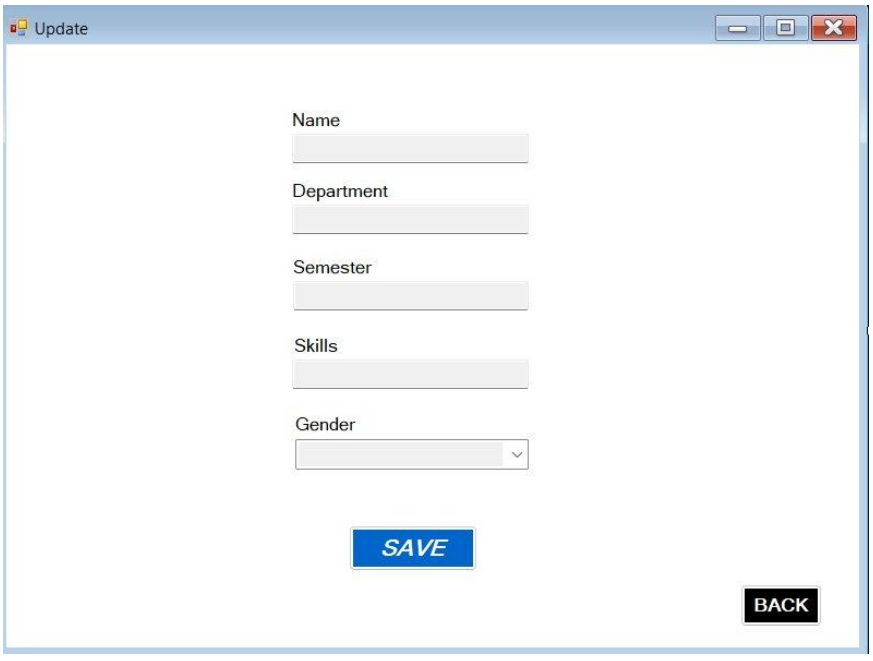

Figure 21: Update Student Information Form

7. Schema Diagram

The database schema diagram below presents the complete relational structure of the PeerSync database, showing all eight tables, their columns with data types, and the foreign key relationships that enforce referential integrity across the system.

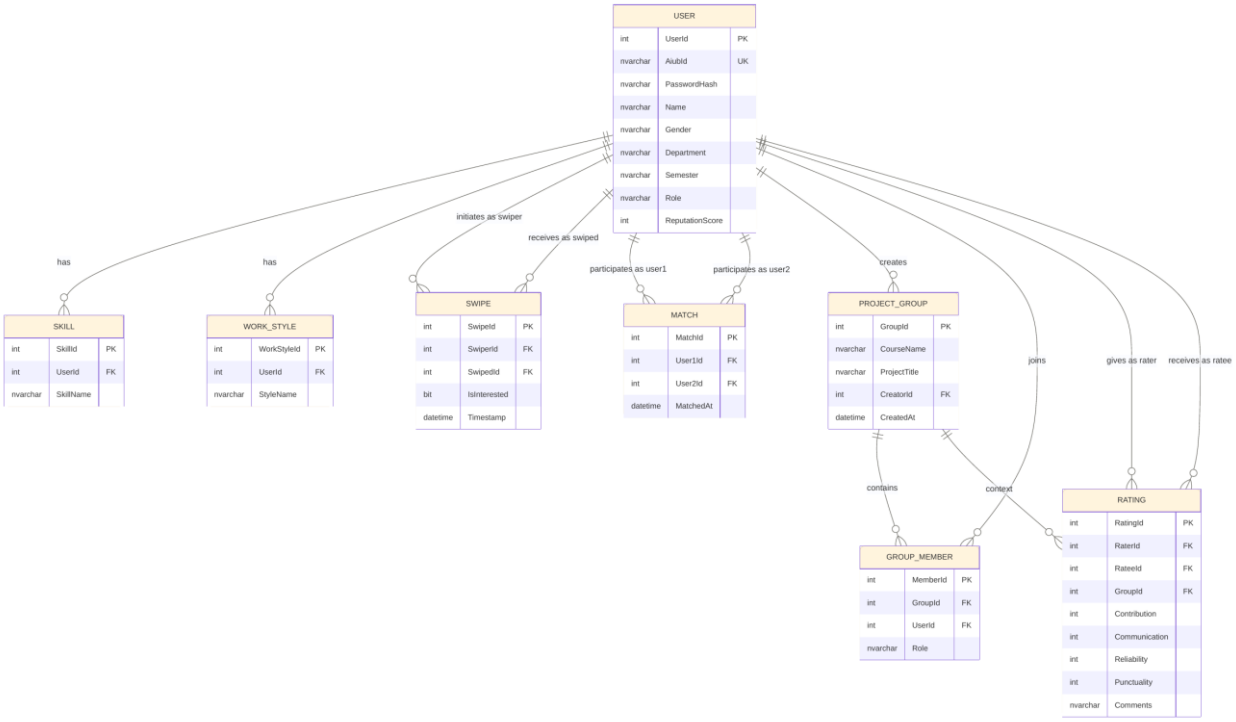

Figure 22: PeerSync Database Schema Diagram

8. Activity Diagrams

The following activity diagrams model the key workflows of the PeerSync application, illustrating the sequence of actions, decision points, and outcomes for each major process.

8.1 Student Registration

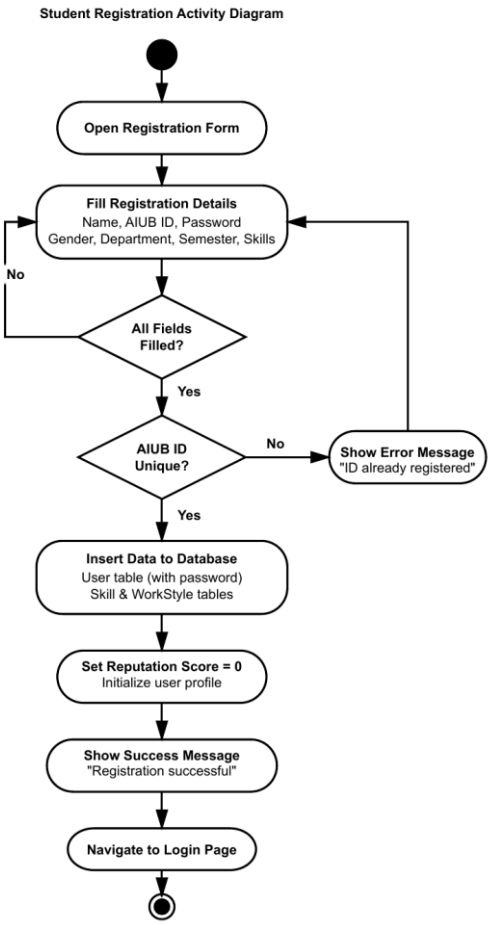

Figure 23: Student Registration Activity Diagram

8.2 Admin User Management

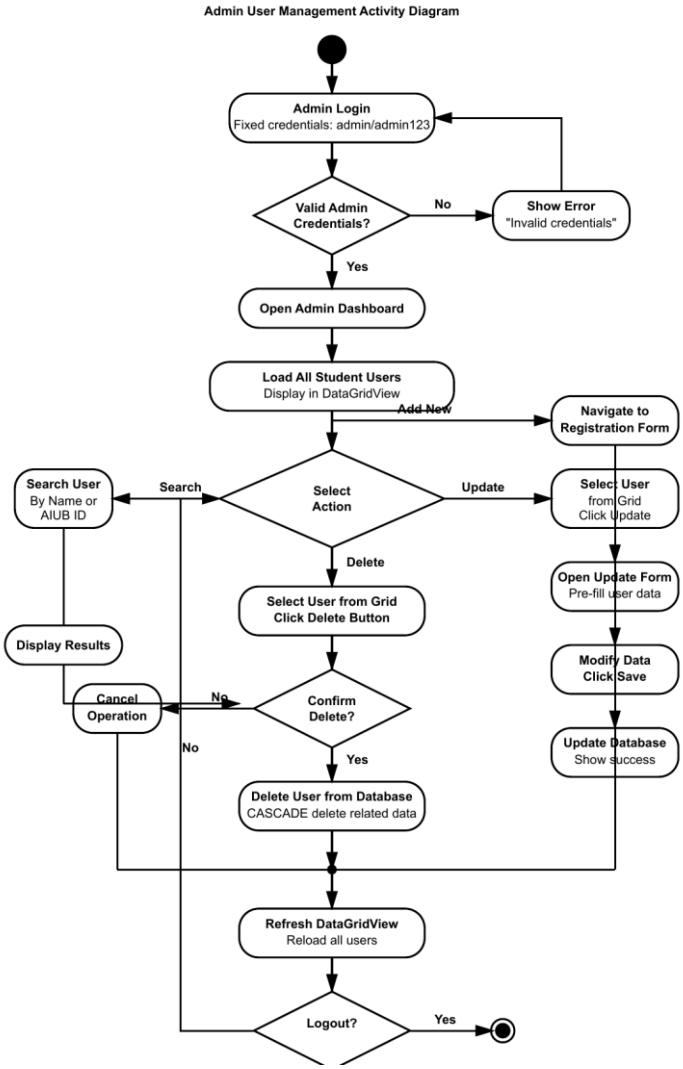

Figure 24: Admin User Management Activity Diagram

8.3 Project Group Creation

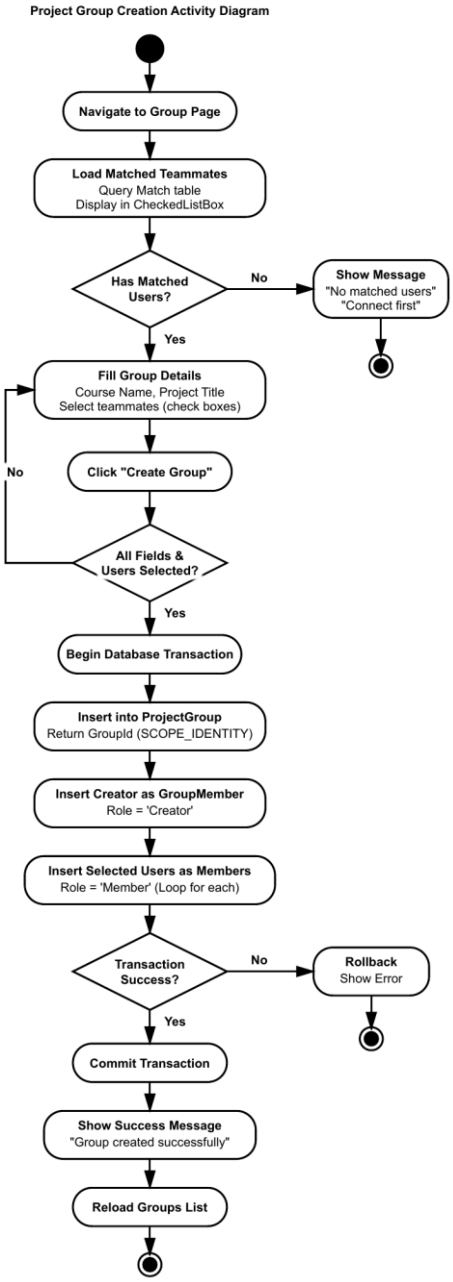

Figure 25: Project Group Creation Activity Diagram

8.4 Student Login & Swipe Matching

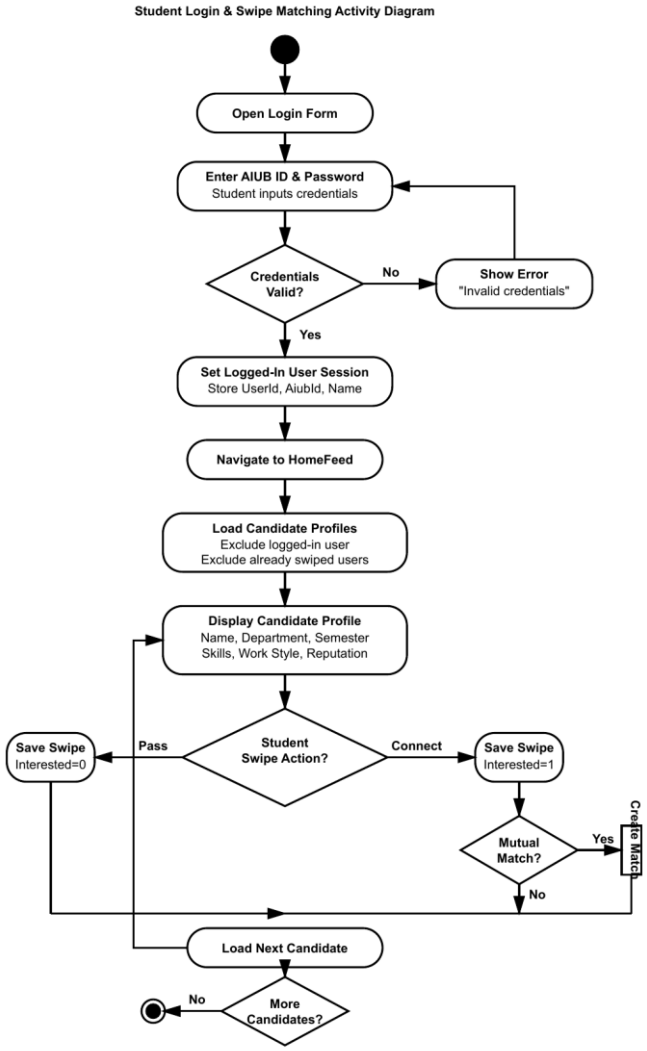

Figure 26: Student Login & Swipe Matching Activity Diagram

8.5 Teammate Rating

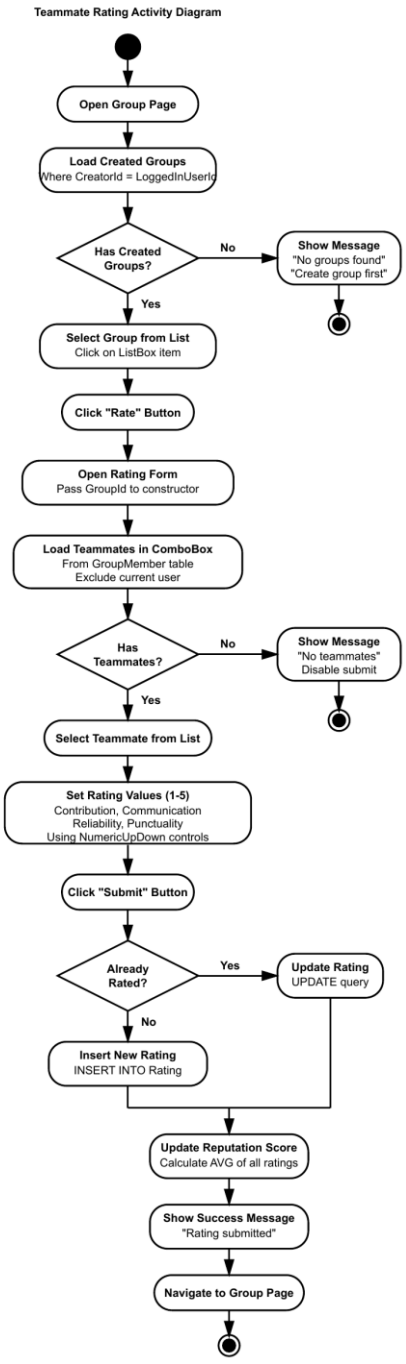

Figure 27: Teammate Rating Activity Diagram

9. Implementation

This section presents the core implementation of PeerSync. Each C# source file is shown with its filename as a header, formatted in a double-column layout. The system is built with C# .NET Windows Forms and ADO.NET for SQL Server database connectivity.

9.1 StudentRegistration.cs

StudentRegistration.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WindowsFormsApp1
{
    public partial class Registration : Form
    {
        public Registration()
        {
            InitializeComponent();
            genderComboBox.Items.Add("Male");
            genderComboBox.Items.Add("Female");
            genderComboBox.Items.Add("Other");
        }




        private void RegistrationButton_Click(object
sender, EventArgs e)
        {
            string connectionString = "data
source=DESKTOP-MJTHEPS\\SQLEXPRESS;


            string name = nameTextBox.Text.Trim();
            string aiubId =
aiubIdTextBox.Text.Trim();
            string password =
passwordTextBox.Text.Trim();
            string gender =
genderComboBox.SelectedItem?.ToString();
            string department =
departmentTextBox.Text.Trim();
            string semester =
semesterTextBox.Text.Trim();
            string skills =
skillsTextBox.Text.Trim();
            string workStyle =
workStyleTextBox.Text.Trim();



            if (string.IsNullOrWhiteSpace(name) ||
string.IsNullOrWhiteSpace(aiu
                string.IsNullOrWhiteSpace(password)
|| string.IsNullOrWhiteSpace
                string.IsNullOrWhiteSpace(department)
|| string.IsNullOrWhiteSpa
            {
                MessageBox.Show("Please fill all
required fields!", "Validation
                return;
            }


            using (SqlConnection connection = new
SqlConnection(connectionString
            {
                try
                {
                    connection.Open();



                    string checkQuery = "SELECT
COUNT(*) FROM [User] WHERE AiubI
                    using (SqlCommand checkCmd = new
SqlCommand(checkQuery, conn
                    {

checkCmd.Parameters.AddWithValue("@AiubId", aiubId);

command.Parameters.AddWithValue("@AiubId", aiubId);

command.Parameters.AddWithValue("@Password",
password);

command.Parameters.AddWithValue("@Name", name);

command.Parameters.AddWithValue("@Gender", gender);

command.Parameters.AddWithValue("@Department",
departmen

command.Parameters.AddWithValue("@Semester",
semester);


                        newUserId =
Convert.ToInt32(command.ExecuteScalar());
                    }



                    if
(!string.IsNullOrWhiteSpace(skills))
                    {
                        string[] skillArray =
skills.Split(',');
                        foreach (string skill in
skillArray)
                        {
                            string skillTrimmed =
skill.Trim();
                            if
(!string.IsNullOrWhiteSpace(skillTrimmed))
                            {
                                string
insertSkillQuery = "INSERT INTO Skill (Us
                                using (SqlCommand
skillCmd = new SqlCommand(inse
                                {

skillCmd.Parameters.AddWithValue("@UserId",

skillCmd.Parameters.AddWithValue("@SkillName

skillCmd.ExecuteNonQuery();
                                }
                            }
                        }
                    }



                    if
(!string.IsNullOrWhiteSpace(workStyle))
                    {
                        string[] styleArray =
workStyle.Split(',');
                        foreach (string style in
styleArray)
                        {
                            string styleTrimmed =
style.Trim();
                            if
(!string.IsNullOrWhiteSpace(styleTrimmed))
                            {
                                string
insertStyleQuery = "INSERT INTO WorkStyle
                                using (SqlCommand
styleCmd = new SqlCommand(inse
                                {

styleCmd.Parameters.AddWithValue("@UserId",

styleCmd.Parameters.AddWithValue("@StyleName

styleCmd.ExecuteNonQuery();
                                }
                            }
                        }
                    }
```

```
                        int count =
(int)checkCmd.ExecuteScalar();

                        if (count > 0)
                        {
                            MessageBox.Show("This
AIUB ID is already registered!
                            return;
                        }
                    }



                    string insertUserQuery = @"INSERT
INTO [User] (AiubId, Passw
                                                VALUES
(@AiubId, @Password, @Name,
                                                SELECT
SCOPE_IDENTITY();";

                    int newUserId;
                    using (SqlCommand command = new
SqlCommand(insertUserQuery,
                    {
```

```
                        MessageBox.Show("Registration
successful! Please login.", "S

                        this.Hide();
                        StudentLogin studentLogin = new
StudentLogin();
                        studentLogin.Show();
                    }
                    catch (Exception ex)
                    {
                        MessageBox.Show("Error: " +
ex.Message, "Database Error", Me
                    }
                }
            }


        private void backButton_Click(object sender,
EventArgs e)
        {
            this.Hide();
            StudentLogin s1 = new StudentLogin();
            s1.Show();


        }
    }
}
```

9.2 StudentLogin.cs

StudentLogin.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static
System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace WindowsFormsApp1
{
    public partial class StudentLogin : Form
    {


        public static int LoggedInUserId = 0;
        public static string LoggedInAiubId = "";
        public static string LoggedInUserName = "";


        private string connectionString = "data
source=DESKTOP-MJTHEPS\\SQLEXPRE


        public StudentLogin()
        {
            InitializeComponent();
        }



        private void loginButton_Click(object sender,
EventArgs e)
        {
            string aiubId =
aiubIDTextBox.Text.Trim();
            string password =
PasswordTextBox.Text.Trim();



            if (string.IsNullOrWhiteSpace(aiubId) ||
string.IsNullOrWhiteSpace(p
            {
                MessageBox.Show("Please fill all
fields!", "Validation Error", M
                return;
            }



            string query = "SELECT UserId, Name FROM
[User] WHERE AiubId = @Aiub
```

```
command.Parameters.AddWithValue("@Password",
password);


                    connection.Open();
                    SqlDataReader reader =
command.ExecuteReader();


                    if (reader.Read())
                    {


                        LoggedInUserId =
Convert.ToInt32(reader["UserId"]);
                        LoggedInAiubId = aiubId;
                        LoggedInUserName =
reader["Name"].ToString();


                        MessageBox.Show("Login
successful!", "Success", MessageB


                        this.Hide();
                        HomeFeed homeFeed = new
HomeFeed();
                        homeFeed.Show();
                    }
                    else
                    {
                        MessageBox.Show("Invalid
credentials!", "Login Failed",
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Error: " +
ex.Message, "Database Error", Messag
            }
        }



        private void backButton_Click(object sender,
EventArgs e)
        {
            this.Hide();
            Welcome welcome = new Welcome();
            welcome.Show();
        }



        private void registrationLabel_Click(object
sender, EventArgs e)
        {
            this.Hide();
```

```
            try
            {
                using (SqlConnection connection = new
SqlConnection(connectionSt
                using (SqlCommand command = new
SqlCommand(query, connection))
                {

command.Parameters.AddWithValue("@AiubId", aiubId);
                Registration registration = new
Registration();
                registration.Show();
            }


        }
    }
```

9.3 Dashboard.cs – LoadAllUsers & Search

Dashboard.cs – LoadAllUsers & searchButton_Click

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace WindowsFormsApp1
{
    public partial class Dashboard : Form
    {

LoadAllUsers();
        }

        private void LoadAllUsers()
        {
            string query =
                "SELECT UserId, AiubId, Name, Gender,
Department, Semester, Repu
                "FROM [User] WHERE Role='Student'";

            using (SqlConnection connection =
                new SqlConnection(connectionString))
            {
                try
                {
                    SqlDataAdapter adapter =
                        new SqlDataAdapter(query,
connection);

                    DataTable dt = new DataTable();

                    adapter.Fill(dt);

                    dataGridView1.DataSource = dt;
                }
                catch (Exception ex)
                {
                    MessageBox.Show(
                        "Error loading data: " +
ex.Message,
                        "Database Error",
                        MessageBoxButtons.OK,
                        MessageBoxIcon.Error);
                }
            }
        }

private void searchButton Click 1(object sender,
EventArgs e)
        {
            string searchText =
                searchTextBox.Text.Trim();

            if
(string.IsNullOrWhiteSpace(searchText))
            {
                LoadAllUsers();
                return;
            }

            string query =
                @"SELECT UserId, AiubId, Name,
Gender,
                Department, Semester, ReputationScore
                FROM [User]
                WHERE Role='Student'
                AND (Name LIKE @Search
                OR AiubId LIKE @Search)";

            using (SqlConnection connection =
                new SqlConnection(connectionString))
            {
                try
                {
                    SqlDataAdapter adapter =
                        new SqlDataAdapter(query,
connection);

adapter.SelectCommand.Parameters.AddWithValue(
                        "@Search",
                        "%" + searchText + "%");

                    DataTable dt = new DataTable();

                    adapter.Fill(dt);

                    dataGridView1.DataSource = dt;

                    if (dt.Rows.Count == 0)
                    {
                        MessageBox.Show(
                            "No users found!",
                            "Search Result",
                            MessageBoxButtons.OK,
MessageBoxIcon.Information);
                    }
                }
                catch (Exception ex)
                {
                    MessageBox.Show(
                        "Error: " + ex.Message,
                        "Database Error",
                        MessageBoxButtons.OK,
                        MessageBoxIcon.Error);
                }
            }
        }
}
```

9.4 Dashboard.cs – deleteButton_Click

Dashboard.cs – deleteButton_Click

```
private void deleteButton_Click_1(object sender,
EventArgs e)
        {
            if (dataGridView1.SelectedRows.Count ==
0)
            {
                MessageBox.Show(
                    "Please select a user to
delete!",
                    "Selection Error",
                    MessageBoxButtons.OK,

ratingCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);


ratingCommand.ExecuteNonQuery();
```

```
                    MessageBoxIcon.Warning);

                return;
            }

            int userId =
                Convert.ToInt32(
                dataGridView1.SelectedRows[0]
                .Cells["UserId"].Value);

            string userName =
                dataGridView1.SelectedRows[0]
                .Cells["Name"].Value.ToString();

            DialogResult result =
                MessageBox.Show(
                    "Are you sure you want to delete
" +
                    userName + "?",
                    "Confirm Delete",
                    MessageBoxButtons.YesNo,
                    MessageBoxIcon.Question);

            if (result == DialogResult.Yes)
            {
                using (SqlConnection connection =
                    new
SqlConnection(connectionString))
                {
                    connection.Open();

                    SqlTransaction transaction =
connection.BeginTransaction();

                    try
                    {

                        string deleteSwipeQuery =
                            "DELETE FROM Swipe " +
                            "WHERE SwiperId=@UserId "
+
                            "OR SwipedId=@UserId";

                        SqlCommand swipeCommand =
                            new SqlCommand(
                                deleteSwipeQuery,
                                connection,
                                transaction);


swipeCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);


swipeCommand.ExecuteNonQuery();



                        string deleteMatchQuery =
                            "DELETE FROM Match " +
                            "WHERE User1Id=@UserId "
+
                            "OR User2Id=@UserId";

                        SqlCommand matchCommand =
                            new SqlCommand(
                                deleteMatchQuery,
                                connection,
                                transaction);


matchCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);


matchCommand.ExecuteNonQuery();



                        string deleteGroupMemberQuery
=
                            "DELETE FROM GroupMember
" +
                            "WHERE UserId=@UserId";

                        SqlCommand gmCommand =
                            new SqlCommand(

                        string deleteProjectQuery =
                            "DELETE FROM ProjectGroup
" +
                            "WHERE
CreatorId=@UserId";

                        SqlCommand pgCommand =
                            new SqlCommand(
                                deleteProjectQuery,
                                connection,
                                transaction);


pgCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);

                        pgCommand.ExecuteNonQuery();



                        string deleteSkillQuery =
                            "DELETE FROM Skill " +
                            "WHERE UserId=@UserId";

                        SqlCommand skillCommand =
                            new SqlCommand(
                                deleteSkillQuery,
                                connection,
                                transaction);


skillCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);


skillCommand.ExecuteNonQuery();



                        string deleteWorkStyleQuery =
                            "DELETE FROM WorkStyle "
+
                            "WHERE UserId=@UserId";

                        SqlCommand wsCommand =
                            new SqlCommand(
                                deleteWorkStyleQuery,
                                connection,
                                transaction);


wsCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);

                        wsCommand.ExecuteNonQuery();



                        string deleteUserQuery =
                            "DELETE FROM [User] " +
                            "WHERE UserId=@UserId";

                        SqlCommand userCommand =
                            new SqlCommand(
                                deleteUserQuery,
                                connection,
                                transaction);


userCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);

                        int rowsAffected =
userCommand.ExecuteNonQuery();

                        transaction.Commit();

                        if (rowsAffected > 0)
                        {
                            MessageBox.Show(
                                "User deleted
successfully!",
                                "Success",
                                MessageBoxButtons.OK,
MessageBoxIcon.Information);
```

```
deleteGroupMemberQuery,
                                connection,
                                transaction);


gmCommand.Parameters.AddWithValue(
                                "@UserId",
                                userId);


                            gmCommand.ExecuteNonQuery();



                            string deleteRatingQuery =
                                "DELETE FROM Rating " +
                                "WHERE GroupId IN " +
                                "(SELECT GroupId FROM
ProjectGroup WHERE CreatorId=@
                                "OR RaterId=@UserId " +
                                "OR RateeId=@UserId";


                            SqlCommand ratingCommand =
                                new SqlCommand(
                                    deleteRatingQuery,
                                    connection,
                                    transaction);
                                    LoadAllUsers();
                                }
                                else
                                {
                                    MessageBox.Show(
                                        "Failed to delete
user!",
                                        "Error",
                                        MessageBoxButtons.OK,
MessageBoxIcon.Error);
                                }
                            }
                            catch (Exception ex)
                            {
                                transaction.Rollback();

                                MessageBox.Show(
                                    "Error: " + ex.Message,
                                    "Database Error",
                                    MessageBoxButtons.OK,
                                    MessageBoxIcon.Error);
                            }
                        }
                    }
                }
```

9.5 Update.cs

Update.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;



namespace WindowsFormsApp1
{
    public partial class Update : Form
    {
        private string connectionString = "data
source=DESKTOP-MJTHEPS\\SQLEXPRE
        private int userId;


        public Update(int userId)
        {
            InitializeComponent();
            this.userId = userId;


            comboBox1.Items.Add("Male");
            comboBox1.Items.Add("Female");
            comboBox1.Items.Add("Other");
        }


        private void Update_Load(object sender,
EventArgs e)
        {
            LoadUserData();
        }



        private void LoadUserData()
        {
            string query = "SELECT * FROM [User]
WHERE UserId = @UserId";


            using (SqlConnection connection = new
SqlConnection(connectionString
            using (SqlCommand command = new
SqlCommand(query, connection))
            {

command.Parameters.AddWithValue("@UserId", userId);


                try
                {
                    connection.Open();
                    SqlDataReader reader =
command.ExecuteReader();
                }
        }



        private void saveButton_Click_1(object
sender, EventArgs e)
        {
            string name = nameTextBox.Text.Trim();
            string department =
departmentTextBox.Text.Trim();
            string semester =
semesterTextBox.Text.Trim();
            string gender = comboBox1.Text;


            string query = @"UPDATE [User] SET
                            Name = @Name,
                            Gender = @Gender,
                            Department = @Department,
                            Semester = @Semester
                            WHERE UserId = @UserId";


            using (SqlConnection connection = new
SqlConnection(connectionString
            using (SqlCommand command = new
SqlCommand(query, connection))
            {

command.Parameters.AddWithValue("@Name", name);

command.Parameters.AddWithValue("@Gender", gender);

command.Parameters.AddWithValue("@Department",
department);

command.Parameters.AddWithValue("@Semester",
semester);

command.Parameters.AddWithValue("@UserId", userId);


                try
                {
                    connection.Open();
                    int rows =
command.ExecuteNonQuery();


                    if (rows > 0)
                    {
                        MessageBox.Show("User updated
successfully!");


                        this.Hide();
                        Dashboard dashboard = new
Dashboard();
                        dashboard.Show();
                    }
                    else
```

```
                    if (reader.Read())
                    {
                        nameTextBox.Text =
reader["Name"].ToString();
                        departmentTextBox.Text =
reader["Department"].ToString()
                        semesterTextBox.Text =
reader["Semester"].ToString();
                        comboBox1.SelectedItem =
reader["Gender"].ToString();
                    }
                }
                catch (Exception ex)
                {
                    MessageBox.Show("Error: " +
ex.Message);
                }
```

```
                    {
                        MessageBox.Show("Update
failed!");
                    }
                }
                catch (Exception ex)
                {
                    MessageBox.Show("Error: " +
ex.Message);
                }
            }
        }

        private void backButton_Click(object sender,
EventArgs e)
        {
            this.Hide();
            Dashboard dashboard = new Dashboard();
            dashboard.Show();
        }
    }
}
```

9.6 HomeFeed.cs – SwipeUser, CheckMutualMatch & CreateMatch

HomeFeed.cs – Core Swipe/Match Logic

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.Common;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static
System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace WindowsFormsApp1
{
    public partial class HomeFeed : Form
    {

private void SwipeUser(bool isInterested)
        {
            if (candidatesTable == null ||
currentIndex >= candidatesTable.Rows.
                return;

            int swipedUserId =
Convert.ToInt32(candidatesTable.Rows[currentIndex

            SaveSwipe(StudentLogin.LoggedInUserId,
swipedUserId, isInterested);

            currentIndex++;
            ShowCurrentCandidate();
        }

private bool CheckMutualMatch(int user1, int user2)
        {
            string query = @"
                SELECT COUNT(*) FROM Swipe
                WHERE SwiperId = @u2 AND SwipedId =
@u1 AND IsInterested = 1";

            using (SqlConnection con = new
SqlConnection(connectionString))
            using (SqlCommand cmd = new
SqlCommand(query, con))
            {
                cmd.Parameters.AddWithValue("@u1",
user1);
                cmd.Parameters.AddWithValue("@u2",
user2);

                con.Open();
                return (int)cmd.ExecuteScalar() > 0;
            }
        }

private void CreateMatch(int user1, int user2)
        {
            string query = "INSERT INTO [Match]
(User1Id, User2Id) VALUES (@u1,

            try
            {
                using (SqlConnection con = new
SqlConnection(connectionString))
                using (SqlCommand cmd = new
SqlCommand(query, con))
                {

cmd.Parameters.AddWithValue("@u1", user1);

cmd.Parameters.AddWithValue("@u2", user2);

                    con.Open();
                    cmd.ExecuteNonQuery();
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Match Error: " +
ex.Message);
            }
        }

private void connectButton_Click_1(object sender,
EventArgs e)
        {
            if (candidatesTable == null ||
currentIndex >= candidatesTable.Rows.
                return;

            int swipedUserId =
Convert.ToInt32(candidatesTable.Rows[currentIndex

            SaveSwipe(StudentLogin.LoggedInUserId,
swipedUserId, true);

            if
(CheckMutualMatch(StudentLogin.LoggedInUserId,
swipedUserId))
            {

CreateMatch(StudentLogin.LoggedInUserId,
swipedUserId);
                MessageBox.Show("It's a Match!");
            }

            currentIndex++;
            ShowCurrentCandidate();
        }
}
```

10. Conclusion

PeerSync is a fully-implemented peer-matching and project team management system for university students. The software tackles the key issues of group project collaboration, including discovering compatible teammates, ensuring accountability, and managing team activities. Matching a simple swipe-based user interface with a reputation management system and a secure database design, PeerSync offers an accessible and efficient platform for academic teamwork.

During every stage of the development the major project goals were attained effectively. The platform makes it possible for learners to set up their profiles, display their skills and work interests, meet appropriate team members through mutual matching, form project teams, and rate each other upon project completion. The scoring system for user reputation builds trust as it shows performance scores that reflect past team experiences. Besides that, the administrator panel supports efficient user management and overall surveillance of the website, hence ensuring harmonious operation and safety.

On top of that, the project was a great learning platform in the fields of software engineering, database design, and application development. We were able to apply the knowledge of database normalization, object-oriented programming, SQL query management, error handling, and user interface design in a real-world context. A few technical difficulties cropped up such as reputation score calculation, match detection logic, and even how to present multiple skills without duplication. These issues were sorted out by having a right database structure, well-optimized queries, and application logic enhancements.

In general, PeerSync serves as an example of how technology can facilitate collaboration and teamwork in higher education settings. The initiative effectively merges social networking ideas with education requirements to come up with a valuable tool for students. Through promoting skill-based matching, responsibility, and giving effective feedback, PeerSync make a good impact on the academic teamwork. Implementing OOP2 concepts through C# Windows Forms and SQL Server integration proficiently shows the practical understanding of the course objectives.

11. GitHub Link

The complete source code for PeerSync is available on GitHub at the following repository:

Repository URL: *[GitHub Repository Link – To Be Added]*

12. References

[1] Microsoft, "C# Language Documentation," Microsoft Docs, 2024. [Online]. Available: https://docs.microsoft.com/en-us/dotnet/csharp/

[2] Microsoft, "SQL Server Technical Documentation," Microsoft Docs, 2024. [Online]. Available: https://docs.microsoft.com/en-us/sql/

[3] Microsoft, "ADO.NET Overview," Microsoft Docs, 2024. [Online]. Available: https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[5] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.

[6] American International University-Bangladesh, "CSC2210 Object Oriented Programming 2 Course Materials," Faculty of Science and Technology, AIUB, Dhaka, Bangladesh, 2025.